Inhaltsverzeichnis

1 Einführung **3**

2 Existierende Produkte **5**

3 Funktionalität und Benutzerschnittstelle **6**
3.1 Searchpane 6
3.1.1 Spektrumsuche 6
3.1.2 Namensuche 7
3.1.3 Formelsuche 7
3.1.4 Struktursuche 7
3.2 Submitpane 8
3.3 Reviewpane 8

4 Datenbankentwurf **9**
4.1 E/R - Diagramm 9
4.2 Relationales Datenbankschema 12

5 Verwendete Software **17**
5.1 MySQL 17
5.1.1 Datendefinition 17
5.1.2 Datenmanipulation 17
5.1.3 UDF 18
5.2 Turbine 19
5.2.1 Torque 19
5.3 Jetspeed 21
5.4 CDK - Chemical Developement Kit 22
5.5 MarvinSketch 23
5.6 NMRShiftDB - Architektur 24

6 Datenbankschnittstelle **25**
6.1 Speicherung der Daten 25
6.2 Suchfunktionen 27
6.2.1 Spektrumsuche 27
6.2.2 Exakte Namensuche 28
6.2.3 Namensuche mittels regulärem Ausdruck 28
6.2.4 Unscharfe Namensuche 29
6.2.5 Exakte Formelsuche 30
6.2.6 Unscharfe Formelsuche 30
6.2.7 Exakte Struktursuche 31
6.2.8 Substruktursuche 32

6.3 Performance der Anfragen 33

7 Zusammenfassung und Ausblick **34**

8 Anhang **34**
8.1 CREATE TABLE Anweisungen 34
8.2 UDF 38
8.3 CREATE INDEX Anweisungen 39

1 Einführung

Kernresonanz (Nuklear-Magnet-Resonanz, NMR)-Spektroskopie wird in der Chemie dazu verwendet, um synthetisierte Verbindungen zu charakterisieren oder die Struktur unbekannter, chemischer Verbindungen aufzuklären. Mit Hilfe des in dieser Arbeit beschriebenen Projektes (i.f. NMRShiftDB genannt) sollen Informationen über organische Verbindungen sowie deren Spektren in einer Datenbank gespeichert und Anwendern via Internet zugänglich gemacht werden. NMRShiftDB wurde von der Chemoinformatikgruppe, unter Leitung von Dr. Christoph Steinbeck am Max-Planck-Institut für chemische Ökologie in Jena, ins Leben gerufen.

Ein Chemiker, der eine unbekannte Struktur isoliert und dessen NMR-Spektrum aufgenommen hat, kann die Datenbank mit Werten, die er aus dem Spektrum gewinnt, durchsuchen. Er erfährt dadurch, ob die Struktur selbst oder ähnliche Strukturen bereits in der Datenbank gespeichert sind und erhält ggf. Aufschluss über die unbekannte Struktur. Neben der Suche nach Spektren besteht außerdem die Möglichkeit chemische Verbindungen durch die Angabe der molekularen Formel, des chemischen Namens oder durch zeichnen einer chemischen Verbindung mit Hilfe eines Strukturediteors zu suchen.

Als Benutzerschnittstelle dient eine Webseite [1], über welche die Datenbank durchsucht werden kann und die Eingabe neuer Spektren erfolgt. NMRShiftDB ist als Open-Source-Projekt angelegt und verwendet deshalb ausschließlich Open-Source-Software. Dies ist ein wesentlicher Unterschied zu bereits existierenden Struktureigenschaftsdatenbanken. Die freie Verfügbarkeit in Forschung und Entwicklung soll es Chemikern ermöglichen ohne Kostenaufwand an Spektrumsinformationen zu gelangen und zum Anwachsen des Datenbestandes beizutragen. Der Open-Source-Charakter von NMRShiftDB stellt außerdem größtmögliche Transparenz her und bietet die Möglichkeit zur Mitarbeit und Erweiterung des Projektes.

In der vorliegenden Studienarbeit soll der grundsätzliche Aufbau von NMRShiftDB beschrieben werden, wobei der Datenbankentwurf sowie die Suchfunktionalitäten auf der Datenbank im Vordergrund stehen. Als Datenbanksystem wird MySQL verwendet (s. 5.1).

NMRShiftDB startet mit einem Datenbestand von etwa 700 Strukturen und 1400 Spektren. Die Zunahme dieses Bestandes ist abhängig vom Erfolg und dem Grad der Bekanntheit des Projektes. Strukturen und Spektren können durch Benutzer, die den Status 'Contributor' besitzen, eingegeben werden. Diesen erhält man durch die Registrierung mit dem Anmeldeformular. Neben dem Contributor gibt es Benutzer, die keine Eingabenrechte besitzen sowie 'Reviewer', die eingegebene Strukturen auf ihre Richtigkeit überprüfen.

Review-Rechte werden von den Administratoren von NMRShiftDB vergeben. Das Projekt NMRShiftDB wird fortwährend weiterentwickelt. Die nachfolgende Beschreibung bezieht sich auf den Stand vom Juni 2002.

Die Arbeit gliedert sich wie folgt. In Kapitel 2 wird ein kurzer Überblick über vergleichbare Projekte und Produkte gegeben. Das folgende Kapitel beschreibt die einzelnen Funktionen sowie die Benutzerschnittstelle. Im vierten Kapitel wird auf den Datenbankentwurf eingegangen. Die verwendete Software wird in Kapitel 5 beschrieben. Kapitel 6 beschäftigt sich mit der Datenbankschnittstelle. Dabei werden die Speicherung der Daten, die Suchanfragen sowie Performanceaspekte behandelt. Im siebten Kapitel erfolgen eine Zusammenfassung sowie ein Ausblick.

2 Existierende Produkte

Struktureigenschaftsdatenbanken sind ein weit verbreitetes und wichtiges Hilfsmittel in der Chemie. Etliche von ihnen sind seit mehr als 20 Jahren im Einsatz und über das Internet erreichbar.

Die am meisten verwendeten Datenbanken sind die des Chemical Abstract Service (CAS) sowie die Beilstein-Datenbank.

Die CAS-Datenbank ist im Wesentlichen eine Referenzdatenbank, d.h. sie verwaltet Artikel aus der Literatur, für die eine Zusammenfassung zur Verfügung gestellt wurde. Die in einem Artikel beschriebenen Strukturen sind gewöhnlich in Strukturdatenbanken mit einer Referenz auf die CAS-Datenbank abgelegt. Damit bietet die CAS-Datenbank eine Verbindung zwischen chemischen Verbindungen und Artikeln, in denen deren Eigenschaften beschrieben sind. CAS stellt eindeutige Nummern für die registrierten Verbindungen bereit, was einen Quasistandard für die Identifizierung solcher Verbindungen darstellt.

Die Beilstein-Datenbank ist eine Strukturdatenbank und verzeichnet die wichtigsten physikalischen Eigenschaften für jede Verbindung, inklusive einer Referenz auf die Primärliteratur.

Spektrendatenbanken existieren nur in geringer Anzahl. Beispiele sind die SpecInfo-, CSEARCH- und die ACDLabs-Datenbank.

Die SpecInfo-Datenbank ist bei der Firma BASF entstanden. Sie enthält chemische Verbindungen sowie verschiedene Typen von Spektren, z.B. NMR und Infrarot.

Die CSEARCH-Datenbank (http://mailbox.univie.ac.at/~robienw8/csearch_server_info.html) wurde ins Leben gerufen von einer Gruppe um Prof. Robien in Wien und entwickelte sich später zu einem kommerziellen Produkt. Sie ist zugänglich über ein E-Mail Interface, mit dem eine chemische Struktur in einem Standardchemiedateiformat an eine E-Mail Adresse gesendet wird. Der Server schickt daraufhin eine Liste von NMR-Daten zurück.

Die ACDLabs-Datenbank (http://www.acdlabs.com) ist ein relativ junges Produkt, jedoch nicht öffentlich zugänglich.

3 Funktionalität und Benutzerschnittstelle

Das wichtigste Merkmal von NMRShiftDB ist seine Suchfunktionalität. Um an die in der Datenbank gespeicherten Informationen zu gelangen, stehen mehrere Suchmöglichkeiten zur Verfügung. Durch die Angabe eines Spektrums, ausgedrückt durch Zahlwerte, können Strukturen mit ähnlichen Spektren gefunden werden. Weitere Suchkriterien sind der chemische Name und die molekulare Formel einer Verbindung. Außerdem besteht die Möglichkeit, durch die Angabe ganzer Strukturen an die gewünschten Informationen zu gelangen.

Ebenso notwendig wie das Suchen ist die Möglichkeit, dem Datenbestand neue Strukturen hinzuzufügen oder vorhandenen Strukturen neue Spektren zuzuordnen. Dabei können außer der Struktur und dem Spektrum noch weitere chemierelevante Daten angegeben werden.

Um eine hohe Qualität der eingegebenen Daten zu gewährleisten, werden diese von ausgewählten Benutzern (Reviewer), die mit besonderen Rechten ausgestattet sind, überprüft. Wenn die Daten korrekt sind, werden sie akzeptiert und stehen dem System zur Verfügung. Anderenfalls werden sie abgelehnt und der Autor erhält eine Nachricht. Eine Update-Möglichkeit besteht im Moment noch nicht, ist aber für die Zukunft denkbar.

Benutzer, die selbst Daten eingeben möchten, müssen sich bei NMRShiftDB registrieren. Die Registrierung erfolgt über ein Anmeldeformular.

Auf der Benutzeroberfläche werden die Hauptfunktionalitäten Search, Submit und Review über Panes angesteuert. Für die einzelnen Panes sind jeweils bestimmte Privilegien erforderlich.

3.1 Searchpane

Das Searchpane besteht aus mehreren Portlets (s. 5.3), in denen die einzelnen Suchmethoden ausgeführt werden können. Diese sind die Spektrumsuche, die Namensuche, die Formelsuche und die Struktursuche.

3.1.1 Spektrumsuche

Bei der Spektrumsuche wird durch die Angabe eines Spektrums nach ähnlichen Spektren gesucht. Ein Spektrum besteht aus mindestens einem Signal. Ein Signal ist ein Wertepaar bestehend aus einem Shiftwert und einem Wert für die Intensität. Der Shiftwert ist ein physikalische Größe, die bei der NMR-Spektroskopie gemessen wird und sich auf ein Atom in einem Molekül bezieht. Die Intensität beschreibt die Stärke des Signals. Das Spektrum wird durch eine Liste von Signalen angegeben. Der Algorithmus zur Berechnung

der Ähnlichkeit zweier Spektren ist vorgegeben und wird hier nicht näher erläutert. Das Suchergebnis enthält eine Liste von Spektren, die dem vorgegebenem Spektrum am ähnlichsten sind.

3.1.2 Namensuche

Hierbei erfolgt die Suche nach einer Verbindung mit Hilfe des chemischen Namens. Für die Angabe des Namens gibt es drei Möglichkeiten. Einmal durch die exakte Form, zum Anderen mit Hilfe eines regulären Ausdrucks. Zum Beispiel werden bei Angabe von 'methyl|hexane' alle Strukturen geliefert, deren chemischer Name die Zeichenkette 'methyl' oder 'hexane' enthalten. Die Auswertung der Zeichenkette erfolgt durch die MySQL-Funktion REGEXP (s. 6.2.3). Als dritte Funktion steht die unscharfe Namensuche zur Verfügung. Sie verfolgt das Ziel Moleküle zu finden, deren Namen ähnlich klingen wie die, der gesuchten Struktur. Diese Funktion erweist sich als nützlich, wenn z.B. Unsicherheiten über die genaue Schreibung eines Namens bestehen. Zur Auswertung der eingegebenen Zeichenkette wird die MySQL-Funktion SOUNDEX verwendet (s. 6.2.4).

3.1.3 Formelsuche

Die Formelsuche bietet die Möglichkeit nach einer konkreten Formel zu suchen oder einen Wertebereich für die Anzahl der in einem Molekül enthaltenen Atome anzugeben. Bsp.: Mit dem Ausdruck 'C5-10H10-20' erhält man alle Moleküle die fünf bis zehn Kohlenstoff-Atome und zehn bis zwanzig Wasserstoff-Atome besitzen. Strukturen, die darüber hinaus noch weitere Atome besitzen, entsprechen nicht dem Suchkriterium.

3.1.4 Struktursuche

Die Struktursuche ermöglicht es, sowohl komplette Strukturen als auch Teilstrukturen zu finden. Für die Suche nach kompletten Strukturen wird ein kanonischer SMILES-String (s. 5.4) verwendet. Dieser wird vom SMILES-String-Generator (s. 5.4) erzeugt und ist für jede gespeicherte Struktur in der Datenbank abgelegt. Die Substruktursuche liefert Moleküle, die eine vorgegebene Teilstruktur enthalten. Für die genaue Funktionsweise siehe 6.2.8.

3.2 Submitpane

Das Submitpane dient der Eingabe von Strukturen und deren Spektren. Zum Editieren der Strukturen steht der Editor MarvinSketch (s. 5.5) zur Verfügung. Dieser liefert eine ASCII-Datei, welche in der Datenbank abgelegt wird und später zum Anzeigen der Struktur verwendet wird. Außer der Struktur muss der Benutzer das Spektrum sowie den Typ des Spektrums angeben. Die Zuordnung der Signale zu den Atomen sowie die Eingabe weiterer chemierelevanter Informationen schließen den Eingabevorgang ab.

3.3 Reviewpane

Nach erfolgreicher Dateneingabe wird ein Reviewer durch eine E-Mail darüber informiert, dass ein neuer Datensatz eingegeben wurde. Durch einen mitgelieferten Schlüssel kann er dann die Daten im Reviewpane kontrollieren und bei Korrektheit freigeben. Erst dann stehen sie dem System zur Verfügung. Ein fehlerhafter Datensatz wird zurückgewiesen und der Autor informiert. Hier wäre eine Korrekturmöglichkeit für den Autor wünschenswert. Diese ist gegenwärtig noch nicht realisiert, ist aber in der weiteren Entwicklung von NMRShiftDB vorstellbar. Die Reviewrechte werden vom Administrator an bestimmte erfahrene Nutzer verliehen, welche für die Qualität der eingegebenen Daten verantwortlich sind.

4 Datenbankentwurf

4.1 E/R - Diagramm

Das E/R-Diagramm (s. Abb. 1) wurde, ausgehend von den chemischen Konzepten wie Molekül, Spektrum, Atom usw., mit dem Ziel entwickelt, den Ausschnitt aus der Chemiewelt, deren Verwaltung NMRShiftDB zur Aufgabe hat, möglichst realitätsnah abzubilden. Die Namen der Entitytypen aus dem E/R-Diagramm stehen im Folgenden in Klammern hinter den deutschen Entsprechungen.

Aufgrund der Eigenschaft von NMRShiftDB als Informationsquelle für NMR-Spektren steht das Spektrum im Mittelpunkt der Betrachtungen. Ein Spektrum ('Spectrum') besitzt eine eindeutige Nummer ('SpectrumID'), ein Eingabedatum ('Date'), eine Prüfmarke ('ReviewFlag'), einen Prüfschlüssel ('ReviewKey'), eine Literaturangabe ('Literature'), eine eindeutige NMRShiftDB-Nummer ('NMRShiftDBNo'), eine Spektrumdatei ('SpecFile') und eine serialisierte Form des Spektrums ('SerializedSpectrum'). Die Prüfmarke gibt an, ob das Spektrum überprüft wurde und ggf. korrekt ist oder zurückgewiesen wurde. Bei Suchanfragen wird dieser Wert überprüft, um auszuschließen, dass Spektren geliefert werden, deren Korrektheit noch nicht kontrolliert wurde. Mit Hilfe des Prüfschlüssels erhält ein Reviewer Zugriff auf das Spektrum. Die Literaturangabe enthält eine Referenz auf das Spektrum in der Literatur. Um das Spektrum auf der Benutzeroberfläche anzeigen zu können wird die Spektrumdatei benötigt. Diese wird durch MarvinSketch (s. 5.5) erzeugt. Für die Zukunft ist die Erzeugung von mehreren Instanzen von NMRShiftDB vorgesehen. Um die Spektren innerhalb dieser Instanzen eindeutig identifizieren zu können, wird die NMRShiftDB-Nummer benötigt. Ein Spektrum hat einen Spektrumtyp und besitzt mindestens ein Signal. Dem Spektrum sind mehrere Bedingungen zugeordnet, unter denen es gemessen bzw. berechnet wurde. Beispiele dafür sind Temperatur, Lösungsmittel usw..

Ein Molekül ('Molecule') hat eine eindeutige Nummer ('MoleculeID'), eine Summenformel ('MolecularFormula'), eine Molare Masse ('MolecularWeight'), einen kanonischen Namen ('CanonicalName'), einen Typ des kanonischen Namens ('CanonicalNameType'), eine bestimmte Anzahl von Elementen ('ElementCount'), eine CAS-Nummer ('CASNumber'), eine NMRShiftDB-Nummer ('NMRShiftDBNumber', siehe Spektrum), eine Moleküldatei ('StructureFile'), eine kanonische Repräsentation ('SMILES-String'), einen Bitstring ('Fingerprint') und eine serialisierte Form des Moleküls ('SerializedMolecule'). Die CAS-Nummer ist eine molekülspezifische Nummer, die vom Chemical Abstract Service [3] vergeben wird und einen Quasi-Standard für die Bezeichnung von chemischen Verbindungen darstellt. Wie die Spek-

trumdatei beim Spektrum dient die Moleküldatei der Darstellung der Molekülstruktur auf der Benutzeroberfläche. Der SMILES-String wird erzeugt durch den SMILES-String-Generator (s. 5.4) und wird für die Struktursuche (s. 6.2.7) verwendet. Ebenfalls für die Struktursuche (s. 6.2.8) wird der Bitstring benötigt, der durch den Fingerprint-Algorithmus (s. 5.4) generiert wird. Ein Molekül besteht aus mindestens zwei Atomen und hat mindestens eine Bindung.

Das von einem Nutzer ('User') eingegebene Spektrum bezieht sich immer auf ein bestimmtes Molekül. Im E/R-Diagramm besteht deshalb eine eindeutige Zuordnung von einem Spektrum auf das zugehörige Molekül. Ein Nutzer hat außerdem die Möglichkeit, Moleküle, Spektrumtypen und Bedingungen anzugeben. Die Tatsache, dass Nutzer in der Datenbank gespeichert sein können, ohne Eingaben gemacht zu haben, wird durch die Kardinalitäten (0,*) an den Beziehungstypen zwischen Nutzer und den genannten Entitytypen ausgedrückt. Die Attribute von Nutzer sind Angaben zur Person und werden im Rahmen des Anmeldevorgangs angegeben.

Eine Bedingung ('Condition') hat einen eindeutigen Typ und Autor. In Abhängigkeit vom Bedingungstyp ('ConditionType') können Spektren entweder drei (bei berechneten Spektren) oder sieben Bedingungen (bei gemessenen Spektren) zugeordnet sein. Die Information, dass ein gemessenes Spektrum z.B. die Temperatur mit dem Wert 20 als Bedingung hat, wird im E/R-Diagramm folgendermaßen ausgedrückt. Das Attribut 'Value' enthält den Wert 20. Diese Bedingung ist einem Bedingungstyp ('ConditionType') zugeordnet. 'ConditionName' würde dann den Wert 'Temperature', 'Conditiontype' ein 'm' für 'measured' und 'DataType' den Wert 'FLOAT' enthalten. Der Grund für diese Modellierung liegt in der Notwendigkeit, zwischen gemessenen und berechneten Spektren unterscheiden zu können.

Der Spektrumtyp ('SpectrumType') gibt Auskunft über die Art des Spektrums (Bsp.: 2-dimensionales C-Spektrum). Diesem ist der schwache Entitytyp Isotop ('Isotope') zugeordnet. Für die Modellierung als schwachen Entitytyp spricht, dass ein Isotop nur in Abhängigkeit von einem Spektrumtyp existiert. Diese Abhängigkeit besteht zwar z.B. auch zwischen Spektrum und Molekül, jedoch steht das Isotop mit keinem weiteren Entitytyp in Beziehung. Die Dimension des Spektrumtyps entspricht der Anzahl von Achen ('Axis') des Isotops. Dieser Zusammenhang stellt eine Integritätsbedingung dar, die ebenfalls bei Signal und Shift besteht.

Ein Signal ist genau einem Spektrum und mindestens einem Atom zugeordnet. Es hat die Intensität und die Dimension als Attribute. Der Shiftwert des Signals ist im Attribut 'Value' abgelegt und bezieht sich auf den zugehörigen Wert im Attribut 'Axis'.

Atome sind durch Bindungen ('Bond') miteinander verbunden und einem

Molekül zugeordnet. Die Kardinalitäten des Beziehungstyps von Atom und Bindung drücken aus, dass ein Atom an ein bis sechs Bindungen und eine Bindung an zwei oder drei Atomen beteiligt sein kann.

Die schwachen Entitytypen, welche dem Molekül zugeordnet sind können jeweils mehrere Einträge enthalten und hätten auch als mengenwertige Attribute im Molekül modelliert werden können. Diese Variante wurde aber, aus Gründen der Übersichtlichkeit und um ein Verstecken der Attribute im Molekül zu verhindern, nicht gewählt. Gleiches gilt für den schwachen Entitytyp 'SpectrumHyperlink'.

Für alle starken Entitytypen wurde eine eindeutige Nummer ('ID') als Primärschlüssel gewählt. Die Primärschlüssel der schwachen Entitytypen setzt sich aus einem ihrer Attribute und dem Fremdschlüssel des Entitytyps von dem sie abhängen zusammen. Die 'Serialized'-Attribute werden im Moment noch nicht verwendet.

4.2 Relationales Datenbankschema

Bei der Umsetzung des E/R-Diagramms in das relationale Datenbankschema wurden alle Entitätstypen als eigenständige Tabellen mit einem eindeutigen Primärschlüssel definiert. Als Primärschlüssel der Tabellen diente bei den starken Entitätstypen der entsprechende Identifizierungsschlüssel des Entitätstyps. Für die Tabellen der schwachen Entitätstypen wurden Kombinationen aus einem Attribut und dem Fremdschlüssel des übergeordneten Entitätstyps gewählt. Die Attribute der Entitätstypen wurden in die korrespondierenden Attribute der Tabellen überführt. Für die n:m-Beziehungstypen wurden ebenfalls eigene Tabellen definiert. Dabei wurden die Identifikationsschlüssel der zugehörigen Entitätstypen als Primärschlüssel gewählt.

Die Schemata für die Nutzerverwaltung von NMRShiftDB (z.B. Tabelle User) werden im Folgenden nicht aufgeführt, da sie Teil von Turbine (s. 5.2) sind. Zum besseren Verständnis wurde jedoch der Entitätstyp 'User' in das E/R-Diagramm übernommen.

Die SQL-Anweisungen zur Erzeugung der Tabellen werden mit Hilfe des objekt-relationalen Datenbanktools Torque (s. 5.2.1) generiert. Dazu wird das relationale Datenbankschema in ein XML-Schema überführt, welches von Torque für die Generierung verwendet wird. Neben den SQL-Anweisungen werden Java-Klassen erzeugt, die dem Zugriff und der Manipulation der Tabellen dienen.

Im Laufe der weiteren Entwicklung von NMRShiftDB kann es durch neue Anforderungen zu Änderungen des Datenbankschemas kommen. Mit Hilfe von Torque lassen sich diese im XML-Schema leicht anpassen und die entsprechenden SQL-Anweisungen erzeugen. Eine manuelle Anpassung der Java-Klassen an die Datenbank ist nicht erforderlich, da diese ebenfalls neu erzeugt werden. Sie entsprechen damit immmer genau dem Datenbank-Schema. Das XML-Schema enhält die Tabellen- und Spaltendefinitionen, Angaben über Primär- und Fremdschlüssel, Datentypen und Nullwerte.

Durch das Datenbankschema werden strukturelle Integritätsbedingungen, die die Konsitenz des Datenbestandes gewährleisten. Die Eindeutigkeitsbedingung ist durch das Vorhandensein eines Identifikationsschlüssels für jede Tabelle erfüllt. Dagegen kann die Wertebereichsbedingung nur in geringem Maße vom Datenbanksystem gewährleistet werden. Die Wertebereiche der Attribute werden durch die Definition geigneter Datentypen soweit wie möglich eingeschränkt. Eine weitere Einschränkung, wie sie z.B. Aufzählungstypen bieten, kann nicht erfolgen, da diese zwar von MySQL unterstützt werden, jedoch mit der aktuellen Version von Torque im XML-Schema nicht angegeben werden können. Gegen eine nachträgliche Änderung der Datentypen in den SQL-Anweisungen spricht, dass dies eine Fehlerquelle bei dem

Zusammenspiel zwischen Java-Klassen und der Datenbank darstellen würde. Außerdem würde dadurch die Verwendung von Torque in Frage gestellt. Ein Attribut, bei dem die Verwendung eines Aufzählungstyps vorteilhaft wäre, ist z.B. ReviewFlag in der Tabelle Spectrum. Als Alternatve wurde der Datentyp CHAR gewählt. Das Attribut ReviewFlag kann die Werte 't'(true), 'f'(false) und 'r'(rejected) annehmen. Aus den genannten Gründen muss die Validierung der Tabelleninhalte im Wesentlichen durch die Programmlogik erfolgen.

Ebenfalls durch die Programmlogik erfolgt die Sicherung der referentiellen Integritätsbedingung. Diese sorgt dafür, dass jeder Wert eines Fremdschlüssels als Wert beim zugehörigen Primärschlüssel vorkommt. Leider werden Fremdschlüssel in der gegenwärtigen Version von MySQL nicht unterstützt. Für die Zukunft ist ein Versionswechsel wünschenswert, der die Sicherung der referentiellen Integrität vom Datenbanksystem ermöglichen könnte.

Angaben über NULL-Werte werden im Datenbankschema nicht ausgedrückt, jedoch im XML-Schema. NULL-Werte sind nur bei den beiden Attributen 'ReviewDate' und 'AssignmentDate' der Tabelle UserSpectrum zulässig. Bei allen anderen Attributen wird die Angabe eines Wertes für notwendig erachtet.

Schema für Tabelle SPECTRUM

Attribut	Typ	Schlüssel	Beschreibung
SPECTRUM_ID	INTEGER	Primärschlüssel	
DATE	DATETIME		Eingabedatum des Spektrums
SPECFILE	MEDIUMTEXT		Stringrepräsentation des Spektrums
SERIALIZED_SPECTRUM	MEDIUMTEXT		Serialisierung
LITERATURE	MEDIUMTEXT		Literaturangabe
REVIEW_FLAG	CHAR		Status der Überprüfung
REVIEW_KEY	VARCHAR (5)		Paßwort für die Überprüfung des Spektrums
NMRSHIFTDB_NUMBER	MEDIUMTEXT	unique	eindeutige Nummer zur Identifizierung eines Spektrums innerhalb mehrerer Instanzen von NMRShiftDB
USER_ID	INTEGER	Fremdschlüssel	referenziert USER
SPECTRUM_TYPE_ID	INTEGER	Fremdschlüssel	referenziert SPECTRUM_TYPE
MOLECULE_ID	INTEGER	Fremdschlüssel	referenziert MOLECULE

Schema für Tabelle CONDITION

Attribut	Typ	Schlüssel	Beschreibung
CONDITION_ID	INTEGER	Primärschlüssel	
VALUE	VARCHAR (255)		Wert der Versuchsbedingung
USER_ID	INTEGER	Fremdschlüssel	referenziert USER
CONDITION_TYPE_ID	INTEGER	Fremdschlüssel	referenziert CONDITION_TYPE

Schema für Tabelle MOLECULE

Attribut	Typ	Schlüssel	Beschreibung
MOLECULE_ID	INTEGER	Primärschlüssel	
DATE	DATETIME		Eingabendatum des Moleküls
STRUCTURE_FILE	MEDIUMTEXT		MDL Repräsentation eines Moleküls
MOLECULAR_FORMULA	VARCHAR (255)		Summenformel
SERIALIZED_MOLECULE	MEDIUMTEXT		Serialisierung
CAS_NUMBER	MEDIUMTEXT	unique	molekülspezifische Nummer, vergeben vom Chemical Abstract Service
NMRSHIFTDB_NUMBER	VARCHAR (255)	unique	eindeutige Nummer zur Identifizierung eines Moleküls innerhalb mehrerer Instanzen von NMRShiftDB
MOLECULAR_WEIGHT	FLOAT		Molekülmasse
FINGERPRINT	MEDIUMTEXT		Stringrepräsentation des Fingerprints
SMILES_STRING	MEDIUMBLOB	unique	SMILES-String-Repräsentation des Moleküls
CANONICAL_NAME	VARCHAR (255)		kanonischer Name
CANONICAL_NAME_TYPE	VARCHAR (255)		Typ des kanonischen Namens
USER_ID	INTEGER	Fremdschlüssel	referenziert USER

Schema für Tabelle CONDITION_TYPE

Attribut	Typ	Schlüssel	Beschreibung
CONDITION_TYPE_ID	INTEGER	Primärschlüssel	
CONDITION_TYPE	CHAR		Typ der Versuchsbedingung
CONDITION_NAME	VARCHAR (255)		Name der Versuchsbedingung
DATA_TYPE	VARCHAR (255)		Datentyp des Wertes der Versuchsbedingung

Schema für Tabelle SPECTRUM_TYPE

Attribut	Typ	Schlüssel	Beschreibung
SPECTRUM_TYPE_ID	INTEGER	Primärschlüssel	
DIMENSIONALITY	INTEGER		Dimension des Spektrums
NAME	VARCHAR (255)		Name des Spektrumtyps
USER_ID	INTEGER	Fremdschlüssel	referenziert USER

Schema für Tabelle SPECTRUM_CONDITION

Attribut	Typ	Schlüssel	Beschreibung
SPECTRUM_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus SPECTRUM
CONDITION_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus CONDITION

Schema für Tabelle ISOTOPE

Attribut	Typ	Schlüssel	Beschreibung
AXIS	INTEGER	Primärschlüssel	Nummer des Isotops
ELEMENT_SYMBOL	VARCHAR (3)		Elementsymbol des Isotops
ATOMIC_MASS	INTEGER		atomare Masse des Isotops
SPECTRUM_TYPE_ID	INTEGER	Fremdschlüssel	referenziert SPECTRUM

Schema für Tabelle SIGNAL

Attribut	Typ	Schlüssel	Beschreibung
SIGNAL_ID	INTEGER	Primärschlüssel	
DIMENSIONALITY	INTEGER		Dimension des Signals
SERIALIZED_SIGNAL	MEDIUMTEXT		Serialisierung
INTENSITY	FLOAT	Intensität des Signals	
SPECTRUM_ID	INTEGER	Fremdschlüssel	referenziert SPECTRUM

Schema für Tabelle SHIFT

Attribut	Typ	Schlüssel	Beschreibung
AXIS	INTEGER	Primärschlüssel	
VALUE	FLOAT		Shiftwert
SIGNAL_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus SIGNAL

Schema für Tabelle ATOM

Attribut	Typ	Schlüssel	Beschreibung
ATOM_ID	INTEGER	Primärschlüssel	
SYMBOL	VARCHAR (3)		Stringrepräsentation des Atoms
SERIALIZED_ATOM	MEDIUMTEXT		Serialisierung
MOLECULE_ID	INTEGER	Fremdschlüssel	referenziert MOLECULE

Schema für Tabelle BOND

Attribut	Typ	Schlüssel	Beschreibung
BOND_ID	INTEGER	Primärschlüssel	
DEGREE	INTEGER		Grad der Bindung
SERIALIZED_BOND	MEDIUMTEXT		Serialisierung
MOLECULE_ID	INTEGER	Fremdschlüssel	referenziert MOLECULE

Schema für Tabelle SIGNAL_ATOM

Attribut	Typ	Schlüssel	Beschreibung
SIGNAL_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus SIGNAL
ATOM_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus ATOM

Schema für Tabelle BOND_ATOM

Attribut	Typ	Schlüssel	Beschreibung
BOND_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus BOND
ATOM_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus ATOM

Schema für Tabelle CHEMICAL_NAME

Attribut	Typ	Schlüssel	Beschreibung
NAME	VARCHAR (255)	Primärschlüssel	chemischer Name des Moleküls
MOLECULE_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus MOLECULE

Schema für Tabelle USER_SPECTRUM

Attribut	Typ	Schlüssel	Beschreibung
USER_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus USER
SPECTRUM_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus SPECTRUM
ASSIGNMENT_DATE	DATETIME		Datum an dem das Spektrum einem Reviewer zugeordnet wurde
REVIEW_DATE	DATETIME		Datum an dem das Spektum überprüft wurde

Schema für Tabelle ATOM_COUNT

Attribut	Typ	Schlüssel	Beschreibung
MOLECULE_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus MOLECULE
SYMBOL	VARCHAR (3)	Primärschlüssel	Symbol eines Elementes aus dem Molekül
COUNT	INTEGER		Anzahl der Vorkommens dieses Elements im Molekül dieses Elements im Molekül

Schema für Tabelle MOLECULE_HYPERLINK

Attribut	Typ	Schlüssel	Beschreibung
MOLECULE_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus MOLECULE
HYPERLINK	MEDIUMTEXT	Primärschlüssel	Hyperlink Referenz
DESCRIPTION	MEDIUMTEXT		Beschreibung des Hyperlinks

Schema für Tabelle SPECTRUM_HYPERLINK

Attribut	Typ	Schlüssel	Beschreibung
SPECTRUM_ID	INTEGER	Primärschlüssel	Fremdschlüssel aus SPECTRUM
HYPERLINK	MEDIUMTEXT	Primärschlüssel	Hyperlink Referenz
DESCRIPTION	MEDIUMTEXT		Beschreibung des Hyperlinks

5 Verwendete Software

5.1 MySQL

Zu Beginn der Arbeit an NMRShiftDB war 3.23.41 die aktuelle Version von MySQL. Sie ist deshalb Grundlage für die nachfolgenden Betrachtungen.

5.1.1 Datendefinition

MySQL erfüllt gegenwärtig noch nicht vollständig die SQL92-Norm, da wesentliche Konzepte im Funktionsumfang fehlen. Relationenschemata mit Attributen , Wertebereichen sowie Primär- und Fremdschlüssel werden mit der Anweisung create table definiert. Als Wertebereiche werden alle numerischen Datentypen von SQL92 sowie die Datums- und Zeit-Typen DATETIME, DATE, TIMESTAMP, TIME und YEAR unterstützt. Die Zeichenkettentypen lauten CHAR, VARCHAR, BLOB, TEXT, ENUM und SET. BLOB und TEXT unterteilen sich jeweils in vier Untertypen sich jedoch nur in der maximalen Länge der Werte, die sie aufnehmen können, unterscheiden. Benutzerdefinierte Wertebereiche können in Gegensatz zu SQL92 nicht eingesetzt werden. Die Klausel not null für den Ausschluss von Nullwerten als Attributwerte wird hingegen unterstützt. Durch die Deklaration eines Primärschlüssels wird implizit ein Index erzeugt. Ein weiterer Unterschied zur SQL92-Norm ist die Möglichkeit Indexe innerhalb der create table-Anweisung anzulegen. Um eine Vermischung von konzeptuelle und interner Ebene zu verhindern, wird dies aber vermieden. Die Angabe von Fremdschlüsseln durch die Klausel foreign key existiert lediglich aus Kompatibilitätsgründen. Sie werden von MySQL gegenwärtig nicht unterstützt. Das gleiche gilt für Sichten.

5.1.2 Datenmanipulation

Zum Auswählen, Einfügen, Ändern und Löschen von Daten besitzt MySQL die üblichen Kommandos aus der SQL-Norm. Gegenwärtig fehlen aber noch einig wichtige Funktionen. So besitzt MySQL keine Sub-Selects. Diese werden nur in der Form INSERT ... SELECT ... unterstützt. Beim Erzeugen von Indexen erlaubt MySQL bei CHAR- und VARCHAR-Spalten eine Längenangabe. Durch CREATE INDEX index_name ON tabelle (spalte(10)) wird ein Index erzeugt, der nur die ersten 10 Zeichen der Spalte benutzt. Als Vorteil dieser Art der Indexerzeugung werden eine kleinere Index-Datei sowie eine mögliche Beschleunigung von INSERT-Operationen angeführt. Für NMRShiftDB wird aber davon kein Gebrauch gemacht. Bei BLOB- und TEXT-

Spalten ist die Längenangabe jedoch erforderlich und läßt sich nicht umgehen (s.7.1).

5.1.3 UDF

Ähnlich wie andere DBMS kann man auch auch MySQL mit einer benutzerdefinierten Funktion (UDF) erweitern. UDFs werden dynamisch mittels CREATE FUNCTION und DROP FUNCTION Anweisungen hinzugefügt bzw. gelöscht. Die Funktion selbst muss in C oder C++ geschrieben sein. NMRShiftDB verwendet zwei UDFs, deren Quellcode in 7.2 zu sehen ist.

5.2 Turbine

Turbine[] ist ein Servlet-basiertes Open-Source-Framework. Es besteht aus einer Reihe von Modulen zur Entwicklung von Web Applikationen und wird für NMRShiftDB in der Version 2.1 verwendet. Als Servlet Container wird Tomcat 3.2 benutzt. Mit Turbine wird die Implementierung des von SUN vorgeschlagenen Modell-View-Controlling(MVC)-Musters verfolgt. Dabei werden alle drei Teile der MVC-Architektur unterstützt. Die Modellerzeugung erfolgt durch das objekt-relationale Tool Torque, das aus einem in XML spezifizierten Datenbankschema die Entitäten erzeugt. Die View wird von Velocity erzeugt, das wie Turbine Teil des Jakarta-Projektes ist. Ein Controller wird von Turbine ebenfalls bereitgestellt. Dieser besteht aus Action-Klassen, die vom Turbine-Servlet aufgerufen werden.

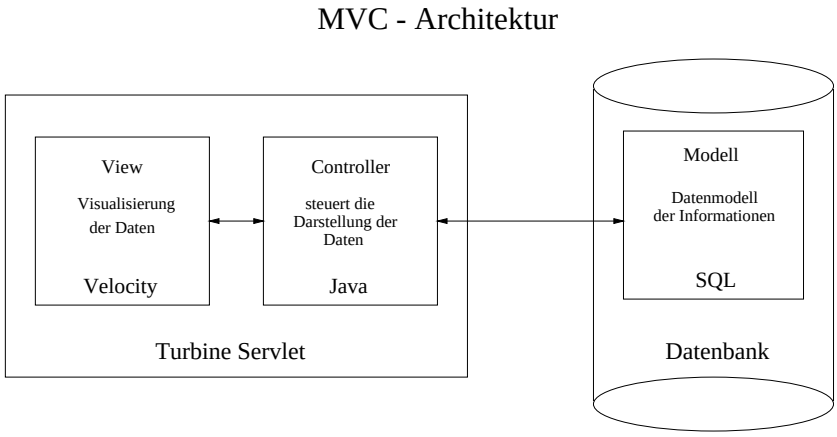


Abbildung 2: MVC-Architektur

Zur Verwaltung der Nutzer von NMRShiftDB bietet Turbine ein Benutzerkonzept, bei dem einem Benutzer mehrere Rollen in verschiedenen Gruppen zugewiesen werden können. Innerhalb von NMRShiftDB sind das die bereits genannten Nutzer Contributor, Reviewer usw..

Außerdem stellt Turbine Pools zur Verwltung von JDBC Verbindungen bereit, die effektive Datenbankzugriffe ermöglichen, sowie einen Id-Broker, der die Id-Verwaltung unterstützt.

5.2.1 Torque

Torque ist ein ein objekt-relationales Datenbanktool, das zur automatischen Erzeugung von Java-Klassen dient. Diese werden mit Hilfe einer datenbankunabhängigen XML-Beschreibung des Datenbankschemas generiert und re-

präsentieren die einzelnen Zeilen und Tabellen. Die SQL-Anweisungen für die physische Erzeugung der Tabellen in der Datenbank werden ebenfalls von Torque generiert.
Bsp.: XML-Schema für den Entitytyp ATOM

```
<table name='ATOM' javaName='DBAtom' idMethod='IdBroker'>
  <column name='ATOM\_ID' required='true' primaryKey='true' type='INTEGER'
  <column name='SYMBOL' required='true' type='VARCHAR' size='3'/>
  <column name='SERIALIZED\_ATOM' required='true' type='LONGVARCHAR'/>
  <column name='MOLECULE\_ID' required='true' type='INTEGER'/>
  <foreign-key foreignTable='MOLECULE'>
    <reference local='MOLECULE\_ID' foreign='MOLECULE\_ID'/>
  </foreign-key>
</table>
```

Die Klassen ermöglichen es Objekte zu erzeugen, zu verändern, zu löschen oder auszuwählen, die einzelne Zeilen in einer Datenbanktabelle darstellen. Das Objektmodell besteht aus 4 Klassen für jede Tabelle im Schema. Für die Tabelle ATOM sind das: Atom, AtomPeer, BaseAtom und BaseAtomPeer. Dabei sind Atom und AtomPeer Subklassen von BaseAtom und BaseAtomPeer. Die beiden Base-Klassen enthalten Torque-generierte Logik und bleiben unverändert. Eigene Programmlogik kann dagegen in den Klassen Atom und AtomPeer hinzugefügt werden. Dies hat den Vorteil, daß bei Änderung des Datenbankschemas nur die Base-Klassen überschrieben werden. Die eigene Programmlogik bleibt jedoch erhalten.

5.3 Jetspeed

Jetspeed [1] ist eine open-source Implementierung eines Informationsportals. Es macht Netzwerkresourcen (Anwendungen, Datenbanken, usw.) für Endbenutzer verfügbar und ist über einen Webbrowser zugreifbar. Mit Hilfe der Jetspeed Portlet APIs können Java-Klassen, sogenannte Portlets, erstellt werden. Diese sind mit Servlets vergleichbar. Die wesentlichen Aufgaben von Jetspeed sind die Benutzer-Authentifizierung, die Personalisierung von Inhalten, das Layout-Management sowie das Logging. Für einige dieser sowie anderer Dienste wie das Pooling von Datenbankverbindungen oder Job-Scheduling nutzt Jetspeed intern das Turbine Framework. Eine Jetspeedseite besteht aus einer Sammlung von Portlets. Die Inhalte der einzelnen Portlets werden durch das Turbine Servlet aggregiert und daraus ein Servlet erzeugt, das ein Servletcontainer zur Anzeige erhält. Die für NMRShiftDB verwendeten Portlets erben von der Klasse AbstractPortlet. Die Methode 'getContent' stellt dabei die zentrale Methode dar. An sie wird ein Objekt der Klasse RunData übergeben. RunData ist die Schnittstelle zur Kommunikation mit dem Portletcontainer und liefert den Zugriff auf HttpRequest- und HttpResponse-Objekte sowie den authentifizierten Benutzer.

5.4 CDK - Chemical Developement Kit

Das CDK ist ein Packet von Java Klassen für Chemoinformatik und Computerchemie.
Es enthält:
- Klassen für Atome, Moleküle und andere chemische Konzepte.
- Hilfsmittel zum Importieren und Exportieren chemischer Strukturen (Moleküle oder Felder von Atomen) durch die Verwendung von Standardchemiedateiformaten
- Hilfsklassen zur Handhabung chemischer Elemente und Isotope
- eine Klasse die eine molekulare Formel analysiert und ein Feld von Knoten zurückgibt
- einen Isomorphismentester der zwei Strukturen miteinander vergleicht und entscheidet ob sie isomorph sind
- einen SMILES (Simplified Molecular Input Line Entry System) Generator, dieser erzeugt einen eindeutigen SMILES String aus einem Molekülobjekt

5.5 MarvinSketch

5.6 NMRShiftDB - Architektur

6 Datenbankschnittstelle

6.1 Speicherung der Daten

Neu eingegebene Strukturen und Spektren werden mit Hilfe der von Torque (s.5.2.1) generierten Klassen in der Datenbank gespeichert. Um z.B. eine Zeile in der Tabelle MOLECULE einzufügen, wird zunächst ein Objekt der Klasse Molecule erzeugt. Die Klasse Molecule besitzt für jedes seiner Attribute eine set-Methode. Dem Objekt werden dann mit Hilfe der set-Methoden die Attributwerte zugeordnet. Die Speicherung des Objekts erfolgt dann mit der save-Methode.
Beispiel:

```
Molecule mol = new Molecule();
mol.setMolecularFormula('C5H10NO');
mol.setElementCount(4);
      ⋮
mol.save();
```

Es ist außerdem möglich die in 5.2.1 erwähnten Peer-Klassen zu verwenden. Die Peer-Klassen besitzen statische Methoden um Operationen auf einer Tabelle auszuführen. Als Alternative zur save-Methode kann man MoleculePeer.doInsert verwenden.
Beispiel:

```
Molecule mol = new Molecule();
mol.setMolecularFormula('C5H10NO');
mol.setElementCount(4);
      ⋮
mol.doInsert(mol);
```

Der Primärschlüssel MoleculeId muss nicht explizit gesetzt werden. Dies wird von Torque selbst übernommen. Für das Auswählen von Zeilen einer Tabelle steht die statische Methode doSelect zur Verfügung. Dieser Methode wird ein sogenanntes Criteria Objekt übergeben, welches das Suchkriterium darstellt. Mit Hilfe des folgenden Codefragments kann man alle Moleküle mit der Summenformel 'C5H10NO' auswählen:

```
Criteria crit = new Criteria();
crit.add(MoleculePeer.MolecularFormula, 'C5H10NO');
```

Vector v = MoleculePeer.doSelect(crit);

Der Vektor v enthält dann die Spaltenelemente der Moleküle, für die die Suchbedingung erfüllt war. Bei der Betrachtung der verwendeten Suchanfragen in 6.2 wird die doSelect-Methode jedoch nicht verwendet, da hierbei immer alle Spalten ausgewählt werden. Es werden aber meistens nur bestimmte Spalten benötigt um z.B. die Struktur in einem Applet anzuzeigen. Deshalb wurden die Select-Anweisung direkt erzeugt.

6.2 Suchfunktionen

6.2.1 Spektrumsuche

Bei der Spektrumsuche sollen diejenigen Moleküle gefunden werden, deren Spektren eine hohe Ähnlichkeit mit dem des eingegebenen Spektrums aufweisen. Dafür wird mit Hilfe der UDF 'fingerprint_compare64' (s.7.2) eine Vorauswahl unter allen Spektren getroffen. Im Attribut FINGERPRINT ist ein Bitstring der Länge 64 abgelegt, dessen Bits Informationen über das Spektrum darstellen. Die Funktion 'fingerprint_compare64' entscheidet durch logische UND-Verknüpfung zweier Bitstrings, ob diese eine gewisse Ähnlichkeit aufweisen. Für alle so ausgewählten Spektren wird dann ein Ähnlichkeitsmaß berechnet und die Strukturen deren Spektren die größte Ähnlichkeit aufweisen, werden angezeigt. Durch diese Vorauswahl vermeidet man die Berechnung der Ähnlichkeiten für alle Spektren. Jedoch läßt sich ein kompletter Tablescan der Tabelle Spektrum bei diesem Verfahren nicht verhindern. Bei der Anfrage werden nur diejenigen Spektren untersucht, die bereits für gültig erklärt wurden. Dies wird im Feld 'REVIEW_FLAG' durch den Wert 't' für 'true' ausgedückt. Mögliche andere Werte sind 'f' für nocht nicht überprüfte Spektren und 'r' für zurückgewiesene Spektren.
Die SQL-Anfrage für die Spektrumsuche lautet:

```
SELECT
        SPECTRUM_ID
FROM
        SPECTRUM
WHERE   fingerprint_compare64(SPECTRUM.FINGERPRINT,
        '00000000000000000000000000001101110110000000000000000000000000') = 'Y' AND
        SPECTRUM.REVIEW_FLAG = 't';
```

6.2.2 Exakte Namensuche

Wenn man den exakten Namen einer chemischen Verbindung kennt, kann man hiermit feststellen, ob sie bereits in der Datenbank vorhanden ist oder Struktur- bzw. Spektrumsinformationen erhalten. Die Eliminierung von Duplikaten durch das Schlüsselwort 'DISTINCT' wird nötig, da einer Struktur mehrere Spektren zugeordnet sein können.
SQL-Anfrage für die Suche nach Ethylcyclohexane:

```
SELECT
        DISTINCT CHEMICAL_NAME.MOLECULE_ID
FROM
        SPECTRUM, CHEMICAL_NAME
WHERE
        CHEMICAL_NAME.NAME = 'ethylcyclohexane' AND
        SPECTRUM.REVIEW_FLAG = 't' AND
        CHEMICAL_NAME.MOLECULE_ID = SPECTRUM.MOLECULE_ID;
```

6.2.3 Namensuche mittels regulärem Ausdruck

Die Angabe des chemischen Namens durch einen regulären Ausdruck bietet vielfältige Möglichkeiten Strukturen mit bestimmten Eigenschaften zu suchen. In der Beispielanfrage werden Verbindungen gesucht, deren Namen mit 'ethylcyclo' beginnen. Dabei wird die MySQL-Funktion 'REGEXP' verwendet, die einen Mustervergleich eines Zeichenkettenausdruckes (Werte aus der Spalte 'NAME') gegen ein Muster durchführt. Bei Übereinstimmung wird 1 zurückgegeben, anderenfalls 0.
Beispielanfrage:

```
SELECT
        DISTINCT CHEMICAL_NAME.MOLECULE_ID
FROM
        CHEMICAL_NAME, SPECTRUM
WHERE
        NAME REGEXP 'ethyl*' AND SPECTRUM.REVIEW_FLAG = 't' AND
        CHEMICAL_NAME.MOLECULE_ID = SPECTRUM.MOLECULE_ID;
```

6.2.4 Unscharfe Namensuche

Aufgrund von unterschiedlichen Schreibweisen chemischer Namen kann es bei der exakten Namensuche zu unbefriedigenden Suchergebnissen kommen. Die unscharfe Namensuche soll dabei helfen die gewünschten Strukturen dennoch zu finden. Hierbei wird die MySQL-Funktion 'SOUNDEX' verwendet, die für im Englischen ähnlich klingende Namen identische soundex-String zurückliefert. In der Beispielanfrage wurde die bereits erwähnte Verbindung Ethylcyclohexane in der Schreibung leicht variiert.
Beispielanfrage:

```
SELECT
        DISTINCT CHEMICAL_NAME.MOLECULE_ID
FROM
        CHEMICAL_NAME, SPECTRUM
WHERE
        SOUNDEX('ethylzyclohexan') = SOUNDEX(NAME) AND
        SPECTRUM.REVIEW_FLAG = 't' AND
        CHEMICAL_NAME.MOLECULE_ID = SPECTRUM.MOLECULE_ID;
```

6.2.5 Exakte Formelsuche

Mit Hilfe der exakten molekularen Formel kann man wie bei der exakten Namensuche ein Molekül direkt suchen. Eine molekulare Formel ist jedoch nicht eindeutig, sodaß für eine Formel mehrere Strukturen gefunden werden können.
Beispielanfrage:

```
SELECT
        DISTINCT MOLECULE.MOLECULE_ID
FROM
        MOLECULE, SPECTRUM
WHERE
        MOLECULE.MOLECULAR_FORMULA = 'C16H15NO2H15' AND
        SPECTRUM.REVIEW_FLAG = 't' AND
        MOLECULE.MOLECULE_ID = SPECTRUM.MOLECULE_ID;
```

6.2.6 Unscharfe Formelsuche

Bei der unscharfen Formelsuche besteht die Möglichkeit Wertebereiche für die Vorkommen bestimmter Elemente in einer chemischen Formel anzugeben. In der Beispielanfrage wird nach allen Molekülen gesucht, die zwischen 5 und 10 Kohlenstoffatome sowie zwischen 10 und 20 Wasserstoffatome besitzen. Dies wird mit der Zeichenkette 'C5-10H10-20' ausgedrückt. Um Moleküle, die zusätzlich zu diesen noch andere Elemente enthalten, auszuschließen, wurde das Attribut 'ELEMENT_COUNT' in der Tabelle 'MOLECULE' eingeführt. Eine Berechnung der Elementanzahl aller Moleküle zur Suchzeit wird somit verhindert.
Beispielanfrage:

```
SELECT
        DISTINCT MOLECULE.MOLECULE_ID
FROM
        SPECTRUM, MOLECULE
        LEFT OUTER JOIN ATOM_COUNT AS C_AC
        ON MOLECULE.MOLECULE_ID = C_AC.MOLECULE_ID
        LEFT OUTER JOIN ATOM_COUNT AS H_AC
        ON MOLECULE.MOLECULE_ID = H_AC.MOLECULE_ID
WHERE
        C_AC.SYMBOL = 'C' AND C_AC.COUNT >= 5 AND C_AC.COUNT <= 10 AND
        H_AC.SYMBOL = 'H' AND H_AC.COUNT >= 10 AND H_AC.COUNT <= 20 AND
        MOLECULE.ELEMENT_COUNT = 2 AND
        SPECTRUM.REVIEW_FLAG = 't' AND
        MOLECULE.MOLECULE_ID = SPECTRUM.MOLECULE_ID;
```

6.2.7 Exakte Struktursuche

Die exakte Struktursuche ermöglicht es herauszufinden, ob ein bestimmtes Molekül bereits in der Datenbank gespeichert ist. Dieses kann über den eindeutigen SMILES-String identifiziert werden. Anders als bei der exakten Formelsuche kann deshalb höchstens eine Struktur gefunden werden.
Beispielanfrage:

```
SELECT
        DISTINCT MOLECULE.MOLECULE_ID
FROM
        MOLECULE, SPECTRUM
WHERE
        MOLECULE.SMILES_STRING = 'C1CCCCC1' AND
        SPECTRUM.REVIEW_FLAG = 't' AND
        SPECTRUM.MOLECULE_ID = MOLECULE.MOLECULE_ID;
```

6.2.8 Substruktursuche

Um zu entscheiden, ob eine gegebene Struktur als Substruktur in einem Molekül existiert, steht im CDK ein Isomorphismentester zur Verfügung. Für einen solchen Test ist jedoch erheblicher Rechenaufwand erforderlich. Um diesen zu verringern, wird eine Vorauswahl unter allen Strukturen getroffen. Zu diesem Zweck wird beim Speichern eines Moleküls ein Bitstring der Länge 1024 durch den Fingerprint-Algorithmus aus dem CDK berechnet und im Feld 'FINGERPRINT' der Tabelle 'MOLECULE' abgelegt. Die gesetzten Bits innerhalb des Bitstrings stellen Charakteristiken der Struktur dar. Um herauszufinden, ob zwei Strukturen gleiche Charakteristiken besitzen, wird eine logische UND-Verknüpfung zweier Bitstrings durchgeführt. Sei bs1 der Fingerprint einer beliebigen Struktur aus der Datenbank und bs2 der Fingerprint der gesuchten Struktur. Wenn die logische Operation bs1 & bs2 wieder bs2 ergibt, so hat die Struktur aus der Datenbank mindestens alle Charakteristiken der Substruktur. Sie stellt somit einen Kandidaten für den Isomorphietest dar. Die UND-Verknüpfung wird durch die UDF 'fingerprint_compare' (s.7.2) durchgeführt. Da die Verknüpfung mit den Fingerprints aller Strukturen in der Datenbank durchgeführt werden muss, ist ein kompletter Tablescan erforderlich, was zu relativ langen Antwortzeiten führt. Typische Suchstrukturen sind Moleküle mit 5 bis 10 Atomen und deren zugehörigen Wasserstoffatomen.
Beispielanfrage:

```
SELECT
        DISTINCT MOLECULE.MOLECULE_ID
FROM
        MOLECULE, SPECTRUM
WHERE
        fingerprint_compare(MOLECULE.FINGERPRINT,
        '10000000001000000010100000101100000011000100000000000000101011000000011000101000000000001 0000000001000100010001000000000000001000000000000000001000011111010010
        1110011000010000000010010100000001000000000000010110000000101000000010000100001010000000110000100001000000000000000000000000000000100000000000000000000000000000000
```

6.3 Performance der Anfragen

Um die zu erwarteten Antwortzeiten bei maximal gefüllter Datenbank zu simulieren, wurden die 700 bereits vorhandenen Strukturen mehrfach dupliziert. Die Anfragen wurden auf etwa 100000 Strukturen sowie die doppelte Anzahl an Spektren ausgeführt. Dabei hatte die Datenbank eine Größe von etwa 400 MB. Die oben aufgeführten Anfragen hatten folgende durchschnittliche Antwortzeiten:

Spektrumsuche: 3496ms

Exakte Namensuche: 6ms

Namensuche mit. reg. Ausdruck: 5153ms

Unscharfe Namesuche: 7596ms

Exakte Formelsuche: 4ms

Unscharfe Formelsuche: 891ms

Exakte Struktursuche: 2ms

Substruktursuche: 28340ms

Die relativ langen Antwortzeiten bei Spektrumsuche, Namensuche m. reg.Ausdruck, unscharfer Namensuche und Substruktursuche resultieren daraus, daß jeweils ein kompletter Tablescan durchgeführt wird. Die Differenz bei Spektrum- und Substruktursuche ist mit der Länge der verwendeten Bitstrings erklärbar. Diese ist bei der Substruktursuche um ein Vielfaches größer als bei der Spektrumsuche.

7 Zusammenfassung und Ausblick

8 Anhang

8.1 CREATE TABLE Anweisungen

```
# ------------------------------------------------------
# CONDITION
# ------------------------------------------------------
drop table if exists CONDITION;
CREATE TABLE CONDITION
(
    CONDITION_ID INTEGER NOT NULL,
    VALUE VARCHAR (255) NOT NULL,
    CONDITION_TYPE_ID INTEGER NOT NULL,
    USER_ID INTEGER NOT NULL,
    PRIMARY KEY(CONDITION_ID),
    FOREIGN KEY (CONDITION_TYPE_ID) REFERENCES CONDITION_TYPE (CONDITION_TYPE_ID),
    FOREIGN KEY (USER_ID) REFERENCES NMRSHIFTDB_USER (USER_ID)
);
# ------------------------------------------------------
# SPECTRUM_CONDITION
# ------------------------------------------------------
drop table if exists SPECTRUM_CONDITION;
CREATE TABLE SPECTRUM_CONDITION
(
    SPECTRUM_ID INTEGER NOT NULL,
    CONDITION_ID INTEGER NOT NULL,
    PRIMARY KEY(SPECTRUM_ID,CONDITION_ID),
    FOREIGN KEY (SPECTRUM_ID) REFERENCES SPECTRUM (SPECTRUM_ID),
    FOREIGN KEY (CONDITION_ID) REFERENCES CONDITION (CONDITION_ID)
);
# ------------------------------------------------------
# CONDITION_TYPE
# ------------------------------------------------------
drop table if exists CONDITION_TYPE;
CREATE TABLE CONDITION_TYPE
(
    CONDITION_TYPE_ID INTEGER NOT NULL,
    CONDITION_NAME VARCHAR (255) NOT NULL,
    CONDITION_TYPE CHAR NOT NULL,
    DATA_TYPE VARCHAR (255) NOT NULL,
    PRIMARY KEY(CONDITION_TYPE_ID)
);
# ------------------------------------------------------
# SPECTRUM_TYPE
# ------------------------------------------------------
drop table if exists SPECTRUM_TYPE;
CREATE TABLE SPECTRUM_TYPE
(
    SPECTRUM_TYPE_ID INTEGER NOT NULL,
    DIMENSIONALITY INTEGER NOT NULL,
    NAME VARCHAR (255) NOT NULL,
    USER_ID INTEGER NOT NULL,
    PRIMARY KEY(SPECTRUM_TYPE_ID),
    FOREIGN KEY (USER_ID) REFERENCES NMRSHIFTDB_USER (USER_ID)
);
# ------------------------------------------------------
# ISOTOPE
# ------------------------------------------------------
drop table if exists ISOTOPE;
CREATE TABLE ISOTOPE
(
    SPECTRUM_TYPE_ID INTEGER NOT NULL,
    AXIS INTEGER NOT NULL,
    ELEMENT_SYMBOL VARCHAR (3) NOT NULL,
    ATOMIC_MASS INTEGER NOT NULL,
    PRIMARY KEY(SPECTRUM_TYPE_ID,AXIS),
    FOREIGN KEY (SPECTRUM_TYPE_ID) REFERENCES SPECTRUM_TYPE (SPECTRUM_TYPE_ID)
);
```

```
# ------------------------------------------------------
# SPECTRUM
# ------------------------------------------------------
drop table if exists SPECTRUM;
CREATE TABLE SPECTRUM
(
    SPECTRUM_ID INTEGER NOT NULL,
    DATE DATETIME NOT NULL,
    SPECFILE MEDIUMTEXT NOT NULL,
    SERIALIZED_SPECTRUM MEDIUMTEXT NOT NULL,
    LITERATURE MEDIUMTEXT NOT NULL,
    REVIEW_FLAG CHAR NOT NULL,
    REVIEW_KEY VARCHAR (6) NOT NULL,
    NMRSHIFTDB_NUMBER MEDIUMTEXT NOT NULL,
    FINGERPRINT MEDIUMTEXT NOT NULL,
    USER_ID INTEGER NOT NULL,
    SPECTRUM_TYPE_ID INTEGER NOT NULL,
    MOLECULE_ID INTEGER NOT NULL,
    PRIMARY KEY(SPECTRUM_ID),
    FOREIGN KEY (USER_ID) REFERENCES NMRSHIFTDB_USER (USER_ID),
    FOREIGN KEY (SPECTRUM_TYPE_ID) REFERENCES SPECTRUM_TYPE (SPECTRUM_TYPE_ID),
    FOREIGN KEY (MOLECULE_ID) REFERENCES MOLECULE (MOLECULE_ID)
);
# ------------------------------------------------------
# SIGNAL
# ------------------------------------------------------
drop table if exists SIGNAL;
CREATE TABLE SIGNAL
(
    SIGNAL_ID INTEGER NOT NULL,
    DIMENSIONALITY INTEGER NOT NULL,
    SERIALIZED_SIGNAL MEDIUMTEXT NOT NULL,
    INTENSITY FLOAT NOT NULL,
    SPECTRUM_ID INTEGER NOT NULL,
    PRIMARY KEY(SIGNAL_ID),
    FOREIGN KEY (SPECTRUM_ID) REFERENCES SPECTRUM (SPECTRUM_ID)
);
# ------------------------------------------------------
# SHIFT
# ------------------------------------------------------
drop table if exists SHIFT;
CREATE TABLE SHIFT
(
    SIGNAL_ID INTEGER NOT NULL,
    AXIS INTEGER NOT NULL,
    VALUE FLOAT NOT NULL,
    PRIMARY KEY(SIGNAL_ID,AXIS),
    FOREIGN KEY (SIGNAL_ID) REFERENCES SIGNAL (SIGNAL_ID)
);
# ------------------------------------------------------
# MOLECULE
# ------------------------------------------------------
drop table if exists MOLECULE;
CREATE TABLE MOLECULE
(
    MOLECULE_ID INTEGER NOT NULL,
    DATE DATETIME NOT NULL,
    STRUCTURE_FILE MEDIUMTEXT NOT NULL,
    MOLECULAR_FORMULA VARCHAR (255) NOT NULL,
    SERIALIZED_MOLECULE MEDIUMTEXT NOT NULL,
    CAS_NUMBER MEDIUMTEXT NOT NULL,
    NMRSHIFTDB_NUMBER VARCHAR (255) NOT NULL,
    MOLECULAR_WEIGHT FLOAT NOT NULL,
    FINGERPRINT MEDIUMTEXT NOT NULL,
    SMILES_STRING MEDIUMBLOB NOT NULL,
    ELEMENT_COUNT INTEGER NOT NULL,
    CANONICAL_NAME VARCHAR (255) NOT NULL,
    CANONICAL_NAME_TYPE VARCHAR (255) NOT NULL,
    USER_ID INTEGER NOT NULL,
    PRIMARY KEY(MOLECULE_ID),
    FOREIGN KEY (USER_ID) REFERENCES NMRSHIFTDB_USER (USER_ID),
);
```

```
# ------------------------------------------------------
# ATOM
# ------------------------------------------------------
drop table if exists ATOM;
CREATE TABLE ATOM
(
    ATOM_ID INTEGER NOT NULL,
    SYMBOL VARCHAR (3) NOT NULL,
    SERIALIZED_ATOM MEDIUMTEXT NOT NULL,
    MOLECULE_ID INTEGER NOT NULL,
    PRIMARY KEY(ATOM_ID),
    FOREIGN KEY (MOLECULE_ID) REFERENCES MOLECULE (MOLECULE_ID)
);
# ------------------------------------------------------
# SIGNAL_ATOM
# ------------------------------------------------------
drop table if exists SIGNAL_ATOM;
CREATE TABLE SIGNAL_ATOM
(
    SIGNAL_ID INTEGER NOT NULL,
    ATOM_ID INTEGER NOT NULL,
    PRIMARY KEY(SIGNAL_ID,ATOM_ID),
    FOREIGN KEY (SIGNAL_ID) REFERENCES SIGNAL (SIGNAL_ID),
    FOREIGN KEY (ATOM_ID) REFERENCES ATOM (ATOM_ID)
);
# ------------------------------------------------------
# BOND
# ------------------------------------------------------
drop table if exists BOND;
CREATE TABLE BOND
(
    BOND_ID INTEGER NOT NULL,
    DEGREE INTEGER NOT NULL,
    SERIALIZED_BOND MEDIUMTEXT NOT NULL,
    MOLECULE_ID INTEGER NOT NULL,
    PRIMARY KEY(BOND_ID),
    FOREIGN KEY (MOLECULE_ID) REFERENCES MOLECULE (MOLECULE_ID)
);
# ------------------------------------------------------
# BOND_ATOM
# ------------------------------------------------------
drop table if exists BOND_ATOM;
CREATE TABLE BOND_ATOM
(
    BOND_ID INTEGER NOT NULL,
    ATOM_ID INTEGER NOT NULL,
    PRIMARY KEY(BOND_ID,ATOM_ID),
    FOREIGN KEY (BOND_ID) REFERENCES BOND (BOND_ID),
    FOREIGN KEY (ATOM_ID) REFERENCES ATOM (ATOM_ID)
);
# ------------------------------------------------------
# CHEMICAL_NAME
# ------------------------------------------------------
drop table if exists CHEMICAL_NAME;
CREATE TABLE CHEMICAL_NAME
(
    MOLECULE_ID INTEGER NOT NULL,
    NAME VARCHAR (255) NOT NULL,
    PRIMARY KEY(MOLECULE_ID,NAME),
    FOREIGN KEY (MOLECULE_ID) REFERENCES MOLECULE (MOLECULE_ID)
);
# ------------------------------------------------------
# USER_SPECTRUM
# ------------------------------------------------------
drop table if exists USER_SPECTRUM;
CREATE TABLE USER_SPECTRUM
(
    USER_ID INTEGER NOT NULL,
    SPECTRUM_ID INTEGER NOT NULL,
    ASSIGNMENT_DATE DATETIME NOT NULL,
    REVIEW_DATE DATETIME,
    PRIMARY KEY(USER_ID,SPECTRUM_ID),
    FOREIGN KEY (USER_ID) REFERENCES NMRSHIFTDB_USER (USER_ID),
    FOREIGN KEY (SPECTRUM_ID) REFERENCES SPECTRUM (SPECTRUM_ID)
);
```

```
# ------------------------------------------
# ATOM_COUNT
# ------------------------------------------
drop table if exists ATOM_COUNT;
CREATE TABLE ATOM_COUNT
(
    MOLECULE_ID INTEGER NOT NULL,
    SYMBOL VARCHAR (3) NOT NULL,
    COUNT INTEGER NOT NULL,
    PRIMARY KEY(MOLECULE_ID,SYMBOL),
    FOREIGN KEY (MOLECULE_ID) REFERENCES MOLECULE (MOLECULE_ID)
);
# ------------------------------------------
# MOLECULE_HYPERLINK
# ------------------------------------------
drop table if exists MOLECULE_HYPERLINK;
CREATE TABLE MOLECULE_HYPERLINK
(
    MOLECULE_ID INTEGER NOT NULL,
    HYPERLINK MEDIUMTEXT NOT NULL,
    DESCRIPTION MEDIUMTEXT NOT NULL,
    PRIMARY KEY(MOLECULE_ID,HYPERLINK),
    FOREIGN KEY (MOLECULE_ID) REFERENCES MOLECULE (MOLECULE_ID)
);
# ------------------------------------------
# SPECTRUM_HYPERLINK
# ------------------------------------------
drop table if exists SPECTRUM_HYPERLINK;
CREATE TABLE SPECTRUM_HYPERLINK
(
    SPECTRUM_ID INTEGER NOT NULL,
    HYPERLINK MEDIUMTEXT NOT NULL,
    DESCRIPTION MEDIUMTEXT NOT NULL,
    PRIMARY KEY(SPECTRUM_ID,HYPERLINK),
    FOREIGN KEY (SPECTRUM_ID) REFERENCES SPECTRUM (SPECTRUM_ID)
);
```

8.2 UDF

```
#ifdef STANDARD
#include <stdio.h>
#include <string.h>
#else
#include <my_global.h>
#include <my_sys.h>
#endif
#include <mysql.h>
#include <m_ctype.h>
#include <m_string.h>
#ifdef HAVE_DLOPEN
extern 'C'
{
    my_bool fingerprint_compare_init(UDF_INIT *initid, UDF_ARGS *args, char *message);
    char *fingerprint_compare( UDF_INIT* initid, UDF_ARGS* args, char *result, unsigned long *length );
    my_bool fingerprint_compare64_init(UDF_INIT *initid, UDF_ARGS *args, char *message);
    char *fingerprint_compare64( UDF_INIT* initid, UDF_ARGS* args, char *result, unsigned long *length );
}
char *fingerprint_compare( UDF_INIT* initid, UDF_ARGS* args, char *result, unsigned long *length )
{
    char *fp1=args->args[0];
    char *fp2=args->args[1];
    for(int i=0;i<1024;i++)
    {
       if(fp1[i] == '0' & fp2[i] == '1' )
       {
          memcpy(result, 'N', 1);
          *length = 1;
          return result;
       }
    }
    memcpy(result, 'Y', 1);
    *length = 1;
    return result;
}
char *fingerprint_compare64( UDF_INIT* initid, UDF_ARGS* args, char *result, unsigned long *length )
{
    char *fp1=args->args[0];
    char *fp2=args->args[1];
    if(fp2[0]=='1' && fp1[0]=='0' && fp1[1]=='0')
    {
       memcpy(result, 'N', 1);
       *length = 1;
       return result;
    }
    if(fp2[63]=='1' && fp1[62]=='0' && fp1[63]=='0')
    {
       memcpy(result, 'N', 1);
       *length = 1;
       return result;
    }
    for(int i=2;i<63;i=i+2)
    {
       if(fp1[i-1] == '0' && fp1[i] == '0' && fp1[i+1] == '0' && fp2[i] == '1')
       {
          memcpy(result, 'N', 1);
          *length = 1;
          return result;
       }
    }
    memcpy(result, 'Y', 1);
    *length = 1;
    return result;
}
#endif /* HAVE_DLOPEN */
```

8.3 CREATE INDEX Anweisungen

```
CREATE INDEX STRUCTURE_FILE_INDEX ON MOLECULE (STRUCTURE_FILE(255));
CREATE INDEX MOLECULAR_FORMULA_INDEX ON MOLECULE (MOLECULAR_FORMULA);
CREATE INDEX ELEMENT_COUNT_INDEX ON MOLECULE (ELEMENT_COUNT);
CREATE UNIQUE INDEX CANONICAL_NAME_INDEX ON MOLECULE (CANONICAL_NAME(255));
CREATE UNIQUE INDEX SMILES_STRING_INDEX ON MOLECULE (SMILES_STRING(255));
CREATE UNIQUE INDEX NMRSHIFTDB_NUMBER_INDEX ON MOLECULE (NMRSHIFTDB_NUMBER);

CREATE INDEX SPECFILE_INDEX ON SPECTRUM (SPECFILE(255));
CREATE INDEX REVIEW_FLAG_INDEX ON SPECTRUM (REVIEW_FLAG);
CREATE UNIQUE INDEX NMRSHIFTDB_NUMBER_INDEX ON SPECTRUM (NMRSHIFTDB_NUMBER);
CREATE INDEX MOLECULE_ID_INDEX ON SPECTRUM (MOLECULE_ID);

CREATE INDEX SYMBOL_INDEX ON ATOM_COUNT (SYMBOL);
CREATE INDEX COUNT_INDEX ON ATOM_COUNT (COUNT);
CREATE INDEX MOLECULE_ID_INDEX ON ATOM_COUNT (MOLECULE_ID);

CREATE INDEX MOLECULE_ID_INDEX ON CHEMICAL_NAME (MOLECULE_ID);
CREATE INDEX NAME_INDEX ON CHEMICAL_NAME (NAME);
```

Literatur

[1] S. Undorf und D. Neumann. Portalentwicklung mit Jetspeed, Javamagazin Juli, 2002.

[2] F. Theis. Turbine - das Apache Web Application Framework, Javamagazin Juni, 2002.

Abbildungsverzeichnis